

Retirement Party Guest Management System

Master Architecture, Database & Step-by-Step Implementation Document

1 GitHub repository (monorepo) • 10 service folders • 1 MongoDB Atlas cluster • Vercel + Render

Next.js • TypeScript • Redux Toolkit • shadcn/ui • Node.js • Express.js • Socket.IO • Firebase Authentication • Gemini • Docker

1. Final Architecture

Use a single GitHub repository (monorepo) containing separate top-level folders for each independently deployable application/service. Microservices architecture describes service boundaries and deployment; folder structure inside one repository is the chosen organization for source control.

GitHub — single repository: retirement-party-guest-management

```
retirement-party-guest-management/  
■■■ retirement-party-frontend-guest  
■■■ retirement-party-frontend-staff  
■■■ retirement-party-frontend-admin  
■■■ retirement-party-api-gateway  
■■■ retirement-party-auth-service  
■■■ retirement-party-registration-service  
■■■ retirement-party-verification-service  
■■■ retirement-party-analytics-service  
■■■ retirement-party-websocket-service  
■■■ retirement-party-admin-assistant
```

Deployment: each Next.js frontend folder deploys to its own Vercel project. Each backend service folder deploys to its own Render Web Service using Render's per-service root directory setting. Database deploys to MongoDB Atlas. Basic Docker containerization and Docker Compose remain for local development, run from the repository root.

Database decision: one MongoDB Atlas cluster and one application database. Do not create one cluster per service.

Authentication decision: use Firebase Authentication for admin/staff sign-in instead of a custom JWT-issuing Auth Service. Firebase issues and manages ID tokens; backend services verify those tokens using the Firebase Admin SDK rather than minting their own JWTs.

2. Service Responsibilities

Service	Responsibility	Main data/dependency
Guest Frontend	Public guest registration and confirmation pages	API Gateway
Staff Frontend	Staff login and QR/phone/confirmation verification UI	API Gateway; Firebase Auth
Admin Frontend	Admin login, dashboard, analytics, assistant UI	API Gateway; Socket.IO; Firebase Auth
API Gateway	Single public API entry point, routing, CORS, auth-token verification middleware, rate limiting	Routes to services
Auth Service	Verifies Firebase ID tokens, syncs user profile and role (RBAC)	Firebase Admin SDK; users
Registration Service	Guest registration, confirmation number, QR token, guest lookup	guests
Verification Service	QR/phone/confirmation verification and atomic check-in	checkins; guest lookup through service boundary
Analytics Service	Dashboard and read-only statistics	Approved guest/check-in data
WebSocket Service	Real-time attendance updates	Attendance events

Service	Responsibility	Main data/dependency
Admin Assistant	Natural-language analytics assistant using Gemini	Analytics Service tools

3. Folder Setup (Single Repository)

Create one GitHub repository and set up these ten top-level folders inside it. Each folder must be independently buildable, testable, Dockerized, and deployable as its own unit, even though all folders live in one repository.

- retirement-party-frontend-guest
- retirement-party-frontend-staff
- retirement-party-frontend-admin
- retirement-party-api-gateway
- retirement-party-auth-service
- retirement-party-registration-service
- retirement-party-verification-service
- retirement-party-analytics-service
- retirement-party-websocket-service
- retirement-party-admin-assistant

Each folder should contain its own package.json, Dockerfile, .env.example, README, and tests. Do not import another service's internal source code across folders. Use REST APIs or event interfaces between services, exactly as if they were separate repositories — the monorepo only changes where the code lives in version control, not how services depend on each other.

Use a root-level .gitignore, root README describing the monorepo layout, and optionally a root package.json with npm/yarn/pnpm workspaces if shared tooling (lint config, TypeScript config) is desired. Shared config is optional; it is not required for the services to function independently.

4. Technology Stack

Layer	Technology
Frontend	Next.js + TypeScript (3 apps: guest, staff, admin)
UI	Tailwind CSS + shadcn/ui
State management	Redux Toolkit
Forms / validation	React Hook Form + Zod
Backend	Node.js + JavaScript + Express.js
Architecture	Microservices + API Gateway (single monorepo, multiple deployable folders)
Database	MongoDB Atlas
MongoDB access	Official MongoDB Node.js Driver; no Mongoose
Authentication	Firebase Authentication (email/password) + RBAC via custom claims / users collection

Layer	Technology
Real-time	WebSockets + Socket.IO
QR scanning	jsQR
QR generation	QR-code library
AI	Gemini API; Admin Assistant only
Containerization	Docker + Docker Compose
CI/CD	GitHub Actions (path-filtered per folder within the single repository)
Deployment	Vercel (3 frontend projects) + Render (7 backend services)

5. MongoDB Atlas Design

Create one MongoDB Atlas cluster and one database named retirement_party.

```
MongoDB Atlas
└─ retirement-party-cluster
   └─ retirement_party
      └─ users
      └─ guests
      └─ checkins
      └─ audit_logs
```

Use the official MongoDB Node.js Driver. Store the Atlas connection string only in environment variables. Atlas requires a database user and network access configuration for application connections.

6. Database Fields

Collect only the information needed for the event. One registration represents one attendee in version 1.

users

Field	Type	Req.	Purpose
_id	ObjectId	Yes	Identifier
firebaseUid	string	Yes	Firebase Authentication UID (unique login identifier)
name	string	Yes	Admin/staff name
email	string	Yes	Email from Firebase Authentication
role	enum	Yes	ADMIN or STAFF
isActive	boolean	Yes	Account enabled/disabled
lastLoginAt	Date/null	No	Last successful login
createdAt	Date	Yes	Created time
updatedAt	Date	Yes	Updated time

Index: unique firebaseUid; unique email. No passwordHash field — Firebase Authentication owns credential storage.

guests

Field	Type	Req.	Purpose
_id	ObjectId	Yes	Guest identifier
name	string	Yes	Guest full name
phone	string	Yes	Phone number
mealPreference	enum	Yes	VEG or NON_VEG
confirmationNumber	string	Yes	Unique human-readable confirmation number
qrTokenHash	string	Yes	Hash of opaque QR token
status	enum	Yes	REGISTERED, CHECKED_IN, CANCELLED
registeredAt	Date	Yes	Registration time
checkedInAt	Date/null	No	Successful check-in time
checkedInBy	ObjectId/null	No	Staff user ID (from users, matched via firebaseUid)
createdAt	Date	Yes	Created time
updatedAt	Date	Yes	Updated time

Indexes: unique confirmationNumber; unique qrTokenHash; phone index; status index; checkedInAt index. Make phone unique only if the business rule is one registration per phone.

7. Check-in & Audit Collections

checkins

Field	Type	Req.	Purpose
_id	ObjectId	Yes	Check-in record
guestId	ObjectId	Yes	Guest identifier
verifiedBy	ObjectId	Yes	Staff user (resolved from Firebase ID token)
method	enum	Yes	QR, PHONE, CONFIRMATION
checkedInAt	Date	Yes	Successful check-in time
createdAt	Date	Yes	Created time
metadata	object/null	No	Small non-sensitive context

Create a unique index on guestId to enforce one successful check-in record per guest.

audit_logs

Field	Type	Req.	Purpose
_id	ObjectId	Yes	Audit identifier
actorUserId	ObjectId/null	No	Admin/staff actor
action	string	Yes	Action name
entityType	string	Yes	USER, GUEST, CHECKIN
entityId	ObjectId/string/ null	No	Affected entity
metadata	object/null	No	Safe contextual information
createdAt	Date	Yes	Event time

8. Data Ownership

- Auth Service — owns users; verifies Firebase ID tokens via Firebase Admin SDK and syncs profile/role
- Registration Service — owns guests
- Verification Service — owns checkins; uses a defined interface for guest lookup/update
- Analytics Service — read-only analytics access
- Admin Assistant — no direct MongoDB access; calls Analytics Service tools
- WebSocket Service — broadcasts approved events; does not own business data

Although the services use one Atlas cluster and live in one repository, maintain logical ownership. Do not let every service freely mutate every collection.

9. API Endpoints

Service	Method	Endpoint	Purpose
Auth	POST	/auth/sync	Verify Firebase ID token; create/update user profile
Auth	GET	/auth/me	Get current user profile and role from verified token
Auth	POST	/auth/revoke	Revoke a user's Firebase refresh tokens (force logout)
Registration	POST	/registrations	Create guest
Registration	GET	/registrations/:confirmationNumber	Find guest
Registration	GET	/registrations/:id	Find guest by ID
Verification	POST	/verification/qr	Verify QR
Verification	POST	/verification/phone	Find by phone
Verification	POST	/verification/confirmation	Find by confirmation
Verification	POST	/verification/check-in	Check in guest
Analytics	GET	/analytics/dashboard	Dashboard statistics
Analytics	GET	/analytics/attendance	Attendance statistics
Analytics	GET	/analytics/meals	Meal statistics
Assistant	POST	/assistant/query	Admin AI question
WebSocket	WS	/socket.io	Real-time connection

All protected endpoints require a valid Firebase ID token in the Authorization header (Authorization: Bearer <idToken>). The API Gateway verifies the token signature and freshness; the Auth Service resolves the associated role for RBAC checks.

10. Core Registration Flow



The registration form collects name, phone number, and meal preference. Attendance status is never accepted from the client. Guest registration does not require Firebase Authentication — it remains a public, unauthenticated flow.

11. Verification & Concurrency Flow

Staff signs in via Firebase Authentication (Staff Frontend)



Staff scans QR / enters phone / confirmation



API Gateway verifies Firebase ID token



Verification Service



Validate input



Resolve guest



Atomic conditional check-in

■■■ already checked in → ALREADY_CHECKED_IN
■■■ not checked in → update + insert checkin



attendance event



WebSocket Service



Admin Frontend dashboard

The check-in operation must be idempotent. A repeated request must not create a second attendance. Use an atomic conditional update and a unique `checkins.guestId` index. MongoDB single-document writes are atomic; use a transaction when multiple documents must change together consistently.

12. Analytics

Metric	Calculation
Total registrations	Count guests that are not CANCELLED
Expected attendees	Count registered guests in v1
Total attended	Count successful check-in records
Attendance percentage	$\text{attended} / \text{expected} \times 100$
Veg registrations	Count <code>mealPreference = VEG</code>
Non-veg registrations	Count <code>mealPreference = NON_VEG</code>
Check-ins over time	Group check-ins by hour/time window

Analytics numbers must be deterministic database results. AI only explains approved results.

13. Admin Assistant AI

Admin (signed in via Firebase Authentication, Admin Frontend)



Admin Assistant Service



Gemini



Select supported analytics tool



Analytics Service



MongoDB



Structured result



Gemini



Natural-language answer

Supported examples: attendance count, vegetarian attendance, non-vegetarian attendance, attendance percentage, registered guests who have not checked in, and check-in trends.

AI restrictions: no direct MongoDB credentials; no arbitrary queries; no guest modifications; no check-in modifications; no authorization decisions. Use predefined read-only tools and validate tool inputs/outputs with Zod. Only ADMIN-role users (verified via Firebase ID token) may call /assistant/query.

14. Frontends & Redux Toolkit

Each of the three frontends is its own Next.js + TypeScript app, living in its own folder inside the monorepo and deployed as its own Vercel project.

retirement-party-frontend-guest/

retirement-party-frontend-guest/

■ app/

■ ■ page.tsx

■ ■ register/

■ ■ confirmation/

■ components/

■ store/

■ services/

■ hooks/

■ lib/

■ types/

retirement-party-frontend-staff/

retirement-party-frontend-staff/

■ app/

■ ■ login/ (Firebase Authentication sign-in)

■ ■ verification/

■ components/

■ store/

■ services/

■ hooks/

```

■■■ lib/                (Firebase client SDK config)
■■■ types/

retirement-party-frontend-admin/
retirement-party-frontend-admin/
■■■ app/
■   ■■■ login/          (Firebase Authentication sign-in)
■   ■■■ dashboard/
■   ■■■ assistant/
■■■ components/
■■■ features/
■■■ store/
■■■ services/
■■■ hooks/
■■■ lib/                (Firebase client SDK config)
■■■ types/
```

Recommended Redux slices: authSlice (holds the Firebase user + role, not tokens themselves — tokens stay with the Firebase SDK), dashboardSlice, verificationSlice, assistantSlice. Keep local form state in React Hook Form instead of placing every input into Redux.

15. Security Rules

Area	Rule
Authentication	Use Firebase Authentication for staff/admin sign-in; never store passwords in MongoDB
ID tokens	Verify every incoming Firebase ID token server-side with the Firebase Admin SDK on each request
RBAC	Enforce ADMIN/STAFF permissions on backend using the role stored in users, keyed by firebaseUid
Validation	Validate every external request with Zod
CORS	Allow only the production frontend origins (guest, staff, admin)
MongoDB	Keep MONGODB_URI in environment variables
Firebase Admin	Keep Firebase Admin SDK service-account credentials in environment variables, never in source
QR	Use unpredictable opaque token; validate server-side
Check-in	Atomic update + unique check-in protection
AI	Read-only approved analytics tools
Secrets	Never commit .env files, for any folder in the repository
Logging	Never log passwords, Firebase ID tokens, service-account keys, or raw QR tokens

16. Backend Service Structure

Each backend service is a folder at the root of the single repository, named retirement-party-<service>, with the following internal layout:

```
retirement-party-<service>/
■■■ src/
■   ■■■ config/
■   ■■■ controllers/
■   ■■■ services/
■   ■■■ routes/
■   ■■■ middleware/      (Firebase Admin token verification lives here)
■   ■■■ validators/
■   ■■■ repositories/
■   ■■■ utils/
■   ■■■ server.ts
■■■ tests/
■■■ .env.example
■■■ .gitignore
■■■ Dockerfile
■■■ package.json
■■■ tsconfig.json
■■■ README.md
```

Use a repository layer around the official MongoDB driver so controllers do not contain database code.

Backend module system: CommonJS (package.json should use "type": "commonjs" or omit the type field; use require()/module.exports).

17. CS / DSA Concepts

Concept	Use in this application
Hashing	Opaque QR tokens, lookup/cache concepts
Searching	Guest lookup by phone, confirmation, QR token
Database indexing	Fast guest verification
Sorting	Admin guest/check-in lists
Sets / duplicate detection	Conceptual duplicate prevention; database remains source of truth
Queues / events	Attendance event flow
Rate limiting	Protect verification endpoints and token-verification middleware
Atomic operations	Safe check-in
Concurrency	Two staff members scan the same guest
Idempotency	Repeated check-in request does not double-count

18. Docker Scope

Keep Docker basic. The purpose is containerization and local multi-service development from the single repository, not Kubernetes.

- Dockerfile in each backend service folder
- One root-level docker-compose.yml referencing each service folder's build context
- docker compose up

Containers:

- api-gateway
- auth-service
- registration-service
- verification-service
- analytics-service
- websocket-service
- admin-assistant-service

MongoDB remains in Atlas. A local MongoDB container is optional and not required for the main setup. The three frontends typically run with `next dev` locally rather than in Docker, since they deploy to Vercel.

19. Exact Implementation Order

- 1 Create one GitHub repository (retirement-party-guest-management) and add the ten top-level service folders.
- 2 Set up the three Next.js frontend folders: retirement-party-frontend-guest, retirement-party-frontend-staff, retirement-party-frontend-admin.
- 3 Set up every Express service folder with JavaScript using CommonJS, health endpoint, error middleware, and environment configuration.
- 4 Create one MongoDB Atlas cluster and retirement_party database.
- 5 Create the MongoDB client/repository layer using the official mongodb package. Do not install Mongoose.
- 6 Create users, guests, checkins, and audit_logs indexes.
- 7 Create a Firebase project and enable Firebase Authentication (email/password) for staff and admin accounts.
- 8 Build Auth Service: verify Firebase ID tokens with the Firebase Admin SDK and sync user role/profile.
- 9 Build Registration Service.
- 10 Build Verification Service and atomic check-in.
- 11 Test Auth (token verification), Registration, and Verification with Postman before building the UI.
- 12 Build API Gateway.
- 13 Build Analytics Service.
- 14 Build WebSocket Service and attendance.updated event.
- 15 Build the guest registration UI (retirement-party-frontend-guest).
- 16 Build the staff verification UI with Firebase sign-in (retirement-party-frontend-staff).
- 17 Build the admin dashboard with Redux Toolkit and Firebase sign-in (retirement-party-frontend-admin).
- 18 Build Admin Assistant with Gemini and read-only analytics tools.
- 19 Add automated tests and concurrency/idempotency tests.
- 20 Add Dockerfiles per backend folder and a root Docker Compose file.
- 21 Deploy each backend folder to its own Render Web Service (set the correct root directory per service) and each frontend folder to its own Vercel project.
- 22 Configure production environment variables, CORS, service URLs, Atlas network access, Firebase Admin credentials, and health checks.
- 23 Run end-to-end production tests.

20. Environment Variables

Service / Folder	Variables
Guest Frontend	NEXT_PUBLIC_API_URL
Staff Frontend	NEXT_PUBLIC_API_URL, NEXT_PUBLIC_WS_URL, NEXT_PUBLIC_FIREBASE_API_KEY, NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN, NEXT_PUBLIC_FIREBASE_PROJECT_ID, NEXT_PUBLIC_FIREBASE_APP_ID
Admin Frontend	NEXT_PUBLIC_API_URL, NEXT_PUBLIC_WS_URL, NEXT_PUBLIC_FIREBASE_API_KEY, NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN, NEXT_PUBLIC_FIREBASE_PROJECT_ID, NEXT_PUBLIC_FIREBASE_APP_ID
API Gateway	PORT, CORS_ORIGIN, AUTH_SERVICE_URL, REGISTRATION_SERVICE_URL, VERIFICATION_SERVICE_URL, ANALYTICS_SERVICE_URL, ASSISTANT_SERVICE_URL, WEBSOCKET_SERVICE_URL, FIREBASE_PROJECT_ID, FIREBASE_CLIENT_EMAIL, FIREBASE_PRIVATE_KEY
Auth	PORT, MONGODB_URI, DB_NAME, FIREBASE_PROJECT_ID, FIREBASE_CLIENT_EMAIL, FIREBASE_PRIVATE_KEY
Registration	PORT, MONGODB_URI, DB_NAME, QR_TOKEN_SECRET
Verification	PORT, MONGODB_URI, DB_NAME, FIREBASE_PROJECT_ID, FIREBASE_CLIENT_EMAIL, FIREBASE_PRIVATE_KEY, REGISTRATION_SERVICE_URL, WEBSOCKET_SERVICE_URL
Analytics	PORT, MONGODB_URI, DB_NAME
WebSocket	PORT, CORS_ORIGIN
Admin Assistant	PORT, GEMINI_API_KEY, ANALYTICS_SERVICE_URL

FIREBASE_PRIVATE_KEY and FIREBASE_CLIENT_EMAIL come from a Firebase service-account JSON, used only by the Firebase Admin SDK on the backend to verify ID tokens. The NEXT_PUBLIC_FIREBASE_* values are the public client-side Firebase config used by the Staff and Admin frontends to sign users in.

21. Render Deployment

Connect the single GitHub repository to Render once. Create one Render Web Service per backend folder, and set each service's Root Directory to that folder (for example retirement-party-auth-service) so Render builds and deploys only that folder while the rest of the monorepo stays untouched. Configure each service's build/start commands and environment variables independently, then deploy.

```
GitHub (single repo)
↓
Render Web Service (root directory = service folder)
↓
Install + build that folder only
↓
Start Express
↓
Service URL
```

Frontends: GitHub (single repo) → Vercel project per frontend folder (root directory set to retirement-party-frontend-guest, retirement-party-frontend-staff, or retirement-party-frontend-admin) → Next.js.

Each Express service should listen on 0.0.0.0 and use the PORT provided by the hosting environment. Services can communicate over Render's private network when configured in the same region.

22. Definition of Done

- ✓ Guest registration works end-to-end.
- ✓ Confirmation number and QR code are generated.
- ✓ Firebase Authentication sign-in works for staff and admin, with RBAC enforced on the backend.
- ✓ Staff can verify using QR, phone, or confirmation number.
- ✓ Concurrent scans cannot double-check-in a guest.
- ✓ Admin dashboard shows correct deterministic statistics.
- ✓ Attendance updates arrive through WebSockets.
- ✓ Admin Assistant answers supported analytics questions.
- ✓ AI cannot modify guest or attendance data.
- ✓ Services have health endpoints and error handling.
- ✓ All backend folders are independently deployable from the single repository.
- ✓ Basic Docker containerization works locally.
- ✓ All three frontends are deployed to Vercel as separate projects.
- ✓ Backend folders are deployed to Render as separate Web Services with per-folder root directories.
- ✓ MongoDB Atlas is securely connected.
- ✓ Core flows are tested.

23. First Milestone — Start Here

Do not start with the dashboard or AI. Finish this milestone first:

- 1 Create one GitHub repository with the ten service folders
- 2 Create a Firebase project and enable email/password Authentication
- 3 Create MongoDB Atlas cluster
- 4 Create retirement_party database
- 5 Create collections + indexes
- 6 Build Auth Service (Firebase ID token verification)
- 7 Build Registration Service
- 8 Build Verification Service
- 9 Test with Postman
- 10 Register one guest
- 11 Check in the guest once
- 12 Attempt a second/concurrent check-in
- 13 Confirm it is rejected safely

When this milestone is complete, the project's core data and transactional foundation is working. Then proceed to the API Gateway, Analytics, WebSockets, the three frontends, AI, Docker, and deployment.

24. Reference Notes

The project planning structure follows the six-document approach from the supplied Vibe Coding guide: PRD, TRD, App Flow, UI/UX Design Brief, Backend Schema, and Implementation Plan. The guide recommends using these documents as the source of truth for an AI coding agent.

Current deployment/database/auth details in this document were checked against official Render, MongoDB, and Firebase documentation: Render supports Node.js/Express web services, GitHub-connected deployment with a configurable root directory (useful for monorepos), Docker runtime, environment variables, health checks, and private service networking; MongoDB documents the official Node.js driver and Atlas connection workflow; Firebase documents client SDKs for sign-in and the Firebase Admin SDK for server-side ID token verification.